PRACTICAL – 8

Implementation of Graph and Searching (DFS and BFS)

Aim:

To implement graph traversal techniques **Depth First Search (DFS)** and **Breadth First Search (BFS)** and traverse all the vertices of a graph.

Objective:

The objective of this experiment is to understand and implement graph traversal techniques using DFS and BFS. DFS explores a graph by going as deep as possible before backtracking, while BFS explores the graph level by level using a queue.

Algorithm:

Depth First Search (DFS)

- Start.
- Read the number of vertices in the graph.
- Read the adjacency matrix of the graph.
- Read the starting vertex.
- Create a visited array and initialize all vertices as unvisited.
- Mark the starting vertex as visited.
- Display the starting vertex.
- For every adjacent vertex:
  If the vertex is not visited, recursively perform DFS on that vertex.
- Continue until all reachable vertices are visited.
- Stop.

Breadth First Search (BFS)

- Start.
- Read the number of vertices in the graph.
- Read the adjacency matrix of the graph.
- Read the starting vertex.
- Create a visited array and initialize all vertices as unvisited.
- Create an empty queue.
- Mark the starting vertex as visited and insert it into the queue.
- Remove a vertex from the queue and display it.

- Check all adjacent vertices.
- If an adjacent vertex is not visited, mark it as visited and insert it into the queue.
- Repeat until the queue becomes empty.
- Stop.

Code:

```
# Implementation of Graph Searching using DFS and BFS

# DFS function
def DFS(graph, vertex, visited):
  visited[vertex] = True
  print(vertex, end=" ")

  for i in range(len(graph)):
    if graph[vertex][i] == 1 and visited[i] == False:
      DFS(graph, i, visited)


# BFS function
def BFS(graph, start):
  visited = [False] * len(graph)
  queue = [start]
  visited[start] = True

  while queue:
    vertex = queue.pop(0)
    print(vertex, end=" ")

    for i in range(len(graph)):
      if graph[vertex][i] == 1 and visited[i] == False:
        visited[i] = True
        queue.append(i)


# Input
n = int(input("Enter the number of vertices: "))

print("Enter the adjacency matrix:")

graph = []
```

```
for i in range(n):
    row = list(map(int, input().split()))
    graph.append(row)

start = int(input("Enter the starting vertex: "))

# DFS
visited = [False] * n

print("\nDFS Traversal:")
DFS(graph, start, visited)

# BFS
print("\nBFS Traversal:")
BFS(graph, start)

print("\nEN.No:9246018683")
```

Output:

```
PS C:\Users\GOUTHAM\OneDrive\Desktop\practical 8> & C:\Users\GOUTHAM\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/GOUTHAM/OneDrive/Desktop/practical 8/DFS_BFS.py"
Enter the number of vertices: 5
Enter the adjacency matrix:
0 1 1 0 0
1 0 0 1 1
0 1 0 0 0
0 0 1 0 0
1 0 0 0 0
Enter the starting vertex: 0

DFS Traversal:
0 1 3 2 4
BFS Traversal:
0 1 2 3 4
EN.No:9246018683
PS C:\Users\GOUTHAM\OneDrive\Desktop\practical 8>
```

Time Complexity:

Case	Time Complexity	Description
Best Case	$O(V^2)$	Checks the adjacency matrix.
Average Case	$O(V^2)$	Checks vertices and connections.
Worst case	$O(V^2)$	Checks all vertices and connections

Space Complexity:

Space Complexity	Description
O(V)	Used for the visited array and recursion stack.